

FitMate React 마이그레이션 — 컴포넌트 트리 & 상태 설계

근거 자료: `react-migration-analysis.md` (페이지/기능 인벤토리, 상태 관리 방식 분석) 이 문서는 그 분석을 바탕으로 실제 React 컴포넌트 트리과 상태 설계를 제안합니다. `Page/Post_detail/vdom/` 은 React 컴포넌트로 완전히 대체되는 것을 전제로 하며, `mini-vdom` 구현 자체 (`createElement/diff/patch/render`)는 설계에 반영하지 않았습니다. 다만 그 구현이 관리하던 **상태**(댓글 배열, 수정/삭제 대상 추적 등)는 그대로 승계하여 아래 설계에 포함했고, 부록에 별도 대응표로 정리했습니다.

0. 설계 전제 — 공용 컴포넌트/혹

8개 페이지 중 `Login`, `Signup` 을 제외한 6개 페이지(`Board`, `Post_detail`, `Post_write`, `Post_edit`, `Profile_edit`, `Password_edit`)가 사이드바 +헤더 마크업을 중복 보유하고 있고, 모달/토스트/폼도 페이지 간 90% 이상 동일합니다(분석 자료 5절). 페이지별 트리에 들어가기 전에, 모든 페이지가 공유할 공용 컴포넌트를 먼저 정의합니다.

공용 요소	대체하는 기존 코드	비고
<code>AppLayout</code>	6개 HTML의 <code>app-layout</code> div 구조	<code><Sidebar></code> + <code><Header></code> + <code>children</code> slot
<code>Sidebar</code>	6개 HTML의 <code><aside class="sidebar"></code>	접힘/펼침 로컬 상태 보유
<code>Header</code> (<code>ProfileDropdown</code> 포함)	<code>profileMenuBtn</code> / <code>dropdownMenu</code> / <code>logoutBtn</code>	로그아웃 로직을 여기 한 곳에만 구현
<code>ConfirmModal</code>	<code>post_delete_modal</code> , <code>comment_delete_modal</code> , <code>withdraw_modal</code>	<code>title</code> / <code>description</code> / <code>onConfirm</code> / <code>onCancel</code> props로 범용화
<code>Toast</code>	<code>Profile_edit.js</code> / <code>Password_edit.js</code> 의 중복 <code>showToast()</code>	<code>useToast()</code> 혹은 전역 <code><ToastProvider></code>
<code>PostForm</code>	<code>Post_write</code> / <code>Post_edit</code> 의 거의 동일한 폼	<code>mode</code> prop (<code>"create"</code> 또는 <code>"edit"</code>)
<code>ValidatedField</code>	각 페이지의 <code>input</code> / <code>helperText</code> 페어 + <code>blur</code> /input 검증	아래 5절에서 상세 비교
<code>validators.js</code> (유틸)	이메일/비밀번호 정규식 4곳 중복	순수 함수로 추출
<code>format.js</code> (유틸)	<code>formatCount</code> / <code>formatDateTime</code> (<code>board.js</code> / <code>post_detail.js</code> 완전 중복)	순수 함수로 추출
<code>useAuth</code> (Context)	<code>localStorage.getItem('accessToken')</code> 산발적 참조	로그인 상태 + <code>accessToken</code> 단일 소스
<code>useInfinitePosts</code>	<code>board.js</code> 의 <code>cursorId</code> / <code>isLoading</code> / <code>IntersectionObserver</code>	게시글 목록 서버 상태 캡슐화
<code>usePostDetail(postId)</code>	<code>post_detail.js</code> 전체(게시글+댓글+좋아요)	서버 상태 + <code>mutation</code> 함수 묶음

1. 페이지별 컴포넌트 트리

1-1. Login (`Page/Login`)

```
LoginPage
└─ LoginForm
   ├── ValidatedField (email)
   ├── ValidatedField (password)
   └─ SubmitButton ("로그인")
```

1-2. Signup (Page/Signup)

```
SignupPage
└─ SignupForm
  │ ProfileImageUploader (미리보기 + 파일 input)
  │ ValidatedField (email)
  │ ValidatedField (password)
  │ ValidatedField (passwordConfirm)
  │ ValidatedField (nickname)
  └─ SubmitButton ("회원가입")
```

1-3. Board (Page/Board)

```
BoardPage
└─ AppLayout
  │ Sidebar
  │ Header
  └─ BoardMain
    │ Greeting + WriteLinkButton
    │ PostList
    │   └─ PostCard × N (post 1개당 1개, key=postId)
    │ InfiniteScrollStatus (로딩/빈 목록/에러 문구)
    └─ ScrollSentinel (IntersectionObserver 타킷)
```

1-4. Post Detail (Page/Post_detail) — mini-vdom 제거 대상

```
PostDetailPage
└─ AppLayout
  │ Sidebar
  │ Header
  └─ PostDetailMain
    │ PostTopBar (뒤로가기, 조회수, 수정/삭제 버튼)
    │ PostHeader (제목, 작성자, 날짜, 이미지, 본문)
    │ PostEngagementRow
    │ │ LikeButton
    │ │ └─ ReportButton
    │ CommentSection
    │ │ CommentForm (등록/수정 겸용, isEditing 여부는 prop으로 전달받음)
    │ │ CommentCountHeading
    │ │ └─ CommentList
    │ │   └─ CommentItem × N (key=commentId, onEdit/onRequestDelete 콜백)
    │ ConfirmModal (게시글 삭제용)
    └─ ConfirmModal (댓글 삭제용)
```

기존 `commentList.addEventListener` 로 자식 버튼 클릭을 위임받아 `data-comment-id` 를 읽던 방식은, `CommentItem` 이 자신의 `comment.commentId` 를 클로저로 이미 알고 있으므로 `onEdit(comment) / onRequestDelete(comment.commentId)` 콜백 호출로 대체됩니다. `data-comment-id` 속성 자체가 필요 없어집니다.

1-5. Post Write / Post Edit (Page/Post_write , Page/Post_edit)

두 페이지는 로직이 90% 이상 동일하므로(분석 자료 5절) 트리도 `mode` prop만 다르게 합성합니다.

```
PostWritePage / PostEditPage
└─ AppLayout
  │ Sidebar
  │ Header
  └─ PostForm (mode="create" | "edit", initialValues?)
    │ ValidatedField (title, 글자수 카운터 포함)
    │ ValidatedField (content)
    │ ImageUploader (파일명 표시 포함)
    └─ SubmitButton
```

1-6. Profile Edit (Page/Profile_edit)

```
ProfileEditPage
└─ AppLayout
  │─ Sidebar
  │─ Header
  │─ ProfileEditForm
  │ │─ ProfileImageUploader (미리보기)
  │ │─ ReadonlyField (email)
  │ │─ ValidatedField (nickname)
  │ │─ SubmitButton
  │─ ConfirmModal (회원탈퇴용)
  └─ Toast
```

1-7. Password Edit (Page/Password_edit)

```
PasswordEditPage
└─ AppLayout
  │─ Sidebar
  │─ Header
  │─ PasswordEditForm
  │ │─ ValidatedField (password)
  │ │─ ValidatedField (passwordConfirm)
  │ │─ SubmitButton
  └─ Toast
```

2. 컴포넌트별 역할 · props · 자체 상태

공용 컴포넌트

컴포넌트	역할	받는 props	자체 상태
AppLayout	사이드바+헤더+본문 레이아웃 뼈대	children	없음 (Sidebar가 접힘 상태 보유)
Sidebar	내비게이션, 접힘/펼침 토글	activeMenu (선택)	collapsed: boolean (로컬)
Header / ProfileDropdown	프로필 메뉴, 로그아웃, 드롭다운	없음 (내부에서 useAuth 사용)	dropdownOpen: boolean (로컬)
ConfirmModal	확인/취소 다이얼로그 공용 틀	open, title, description, onConfirm, onCancel	없음 (완전 제어 컴포넌트)
Toast	완료 메시지 잠깐 표시	message, visible (또는 useToast() 가 반환하는 값 사용)	표시 여부는 훅에서 관리, 컴포넌트는 무상태
PostForm	게시글 작성/수정 공용 폼	mode, initialValues?, onSubmit, submitting	values (title/content/image), errors (로컬)
ValidatedField	input + helper text 쌍	label, value, error, onChange, onBlur	없음 (제어 컴포넌트, 값/에러는 부모가 소유)
ProfileImageUploader	프로필 이미지 선택 + 미리보기	previewUrl, onFileChange, error	없음 (미리보기 URL은 부모의 values에서 계산)

Board 페이지

컴포넌트	역할	props	자체 상태
BoardPage	게시글 목록 조회, useInfinitePosts 사용	없음	없음 (훅에 위임)
PostList	게시글 카드 목록 렌더링	posts: Post[]	없음

컴포넌트	역할	props	자체 상태
PostCard	게시글 카드 1건 (썸네일/제목/통계/작성자)	post: Post	없음 (순수 표시 + 상세 페이지 링크)
InfiniteScrollStatus	로딩/빈목록/에러 문구	status ('loading' / 'empty' / 'error' / 'idle')	없음
ScrollSentinel	IntersectionObserver 관측 대상	onIntersect	없음 (내부에서 ref + useEffect 로 observer 등록)

Post Detail 페이지

컴포넌트	역할	props	자체 상태
PostDetailPage	usePostDetail(postId) 혹은로 서버 상태/무테이션 소유, 하위에 전달	URL의 postId	없음 (후에 위임)
PostHeader	제목/작성자/날짜/이미지/본문 표시	post	없음
PostTopBar	조회수, 수정/삭제 버튼(작성자만)	viewCount, isOwner, onEdit, onRequestDelete	없음
LikeButton	좋아요 토글	liked, likeCount, onToggle	없음 (완전 제어)
ReportButton	신고 실행	onReport	없음
CommentSection	댓글 영역 조립	comments, editingComment, 각종 콜백	없음
CommentForm	댓글 작성/수정 검증 입력창	editingComment (null이면 작성 모드), onSubmit, onCancelEdit	draft: string (입력 중인 텍스트, 로컬)
CommentList	댓글 목록 렌더링	comments, onEdit, onRequestDelete	없음
CommentItem	댓글 1건 (작성자/날짜/본문/수정·삭제 버튼)	comment, onEdit, onRequestDelete	없음

Post Write / Edit 페이지

컴포넌트	역할	props	자체 상태
PostWritePage	작성 API 호출, 성공 시 이동	없음	없음
PostEditPage	기존 값 조회(prefill) + 수정 API 호출	URL의 postId	initialValues (서버에서 받아온 값, 로컬에 보관 후 PostForm 에 전달)
PostForm	(위 표와 동일)		

Profile Edit / Password Edit 페이지

컴포넌트	역할	props	자체 상태
ProfileEditPage	회원 정보 조회 + 수정/탈퇴 API 호출	없음	user (서버 상태), 탈퇴 modal open 여부
ProfileEditForm	닉네임/프로필사진 수정 폼	initialValues, onSubmit	values, errors (로컬)
PasswordEditPage	비밀번호 변경 API 호출	없음	없음 (폼이 상태 보유)
PasswordEditForm	비밀번호/확인 입력	onSubmit	values, errors (로컬)

Login / Signup 페이지

컴포넌트	역할	props	자체 상태
LoginPage	로그인 API 호출, 성공 시 이동	없음	없음
LoginForm	이메일/비번 입력 + 검증	onSubmit	values, errors (로컬)
SignupPage	회원가입 API 호출, 409 에러 필드 매핑	없음	없음
SignupForm	5개 필드 입력 + 검증	onSubmit, serverErrors? (409 응답 필드)	values, errors (로컬)

3. 상태 분류 — 로컬 / 공유(상위 끌어올림, 전역) / 서버

서버 상태 (Server State) — API 응답을 그대로 보관, 최신성을 서버가 결정

상태	위치(혹/컴포넌트)	기존 대응
게시글 목록(cursor 페이지네이션)	<code>useInfinitePosts</code> → <code>BoardPage</code>	<code>board.js</code> 전역 배열 없음(즉시 DOM 반영) → React에서는 <code>posts</code> state로 승격 필요
게시글 상세(<code>post</code>)	<code>usePostDetail</code> → <code>PostDetailPage</code>	<code>post_detail.js</code> <code>DOMContentLoaded</code> 에서 필드별 <code>textContent</code> 대입
댓글 목록(<code>comments</code>)	<code>usePostDetail</code> → <code>PostDetailPage</code>	<code>post_detail.js:127</code> 전역 <code>let comments = []</code> (vdom diff용) — 그대로 승계, 자세한 내용은 부록 참고
회원 정보(<code>user</code>)	<code>ProfileEditPage</code>	<code>profile_edit.js:76-94</code> <code>getUser()</code>
게시글 수정 진입 시 기존 값	<code>PostEditPage</code>	<code>post_edit.js:66-82</code> <code>defaultEditPage()</code>
인증 사용자(<code>accessToken</code> 유효성)	<code>AuthContext</code> (<code>useAuth</code>)	<code>localStorage.getItem('accessToken')</code> 산발적 참조

공유 상태 (상위 컴포넌트로 끌어올림 / 전역)

상태	끌어올리는 곳	이유
<code>accessToken</code> / 로그인 여부	전역 <code>AuthContext</code>	모든 페이지·모든 API 호출이 참조. <code>Header</code> 의 로그아웃도 여기서 초기화해야 6개 페이지에서 동일하게 동작(기존 버그 수정, 5절 참고)
<code>editingComment</code> (수정 대상 댓글)	<code>PostDetailPage</code> (또는 <code>usePostDetail</code>)	<code>CommentItem</code> (수정 버튼)이 트리거하고 <code>CommentForm</code> (입력값 프리필+버튼 라벨)이 소비 — 두 형제 컴포넌트가 공유해야 하므로 부모로 끌어올림. 기존 <code>currentEditCommentId</code> / <code>currentEditCommentBody</code> 전역 변수에 대응
<code>deleteTargetCommentId</code>	<code>PostDetailPage</code>	<code>CommentItem</code> (삭제 버튼)이 설정, <code>ConfirmModal</code> (확인 시 실제 삭제 API 호출)이 소비. 기존 <code>currentDeleteCommentId</code> / <code>currentDeleteItem</code> 전역 변수에 대응
<code>isLiked</code> / <code>likeCount</code>	<code>PostDetailPage</code> (<code>usePostDetail</code> 내부)	최초값은 게시글 상세 조회 응답(<code>is_liked</code> , <code>like_count</code>)에서 오고, <code>LikeButton</code> 클릭 시 API 응답으로만 갱신 — 서버 상태와 상호작용하는 파생 상태라 페이지 레벨에서 관리
<code>commentCount</code>	<code>PostDetailPage</code> (<code>usePostDetail</code> 내부)	댓글 생성/삭제 API 응답에 포함되어 내려오며, <code>CommentCountHeading</code> 이 그 값을 그대로 표시
게시글 삭제 모달 <code>open</code> 여부	<code>PostDetailPage</code>	<code>PostTopBar</code> (열기 트리거)와 <code>ConfirmModal</code> (닫기/확인)이 공유
회원탈퇴 모달 <code>open</code> 여부	<code>ProfileEditPage</code>	위와 동일 패턴

로컬 상태 (컴포넌트 자체 상태로 충분한 것)

상태	위치	비고
사이드바 접힘/펼침	<code>Sidebar</code>	다른 컴포넌트가 참조하지 않음. 페이지 이동 시 초기화되는 기존 동작과 동일
프로필 드롭다운 열림	<code>Header</code>	위와 동일
폼 입력값(<code>values</code>)·필드 에러(<code>errors</code>)	각 <code>*Form</code> 컴포넌트	제출 시점에만 부모(Page)로 전달(<code>onSubmit(values)</code>). 필드별 <code>isValidX</code> 불리언을 따로 두지 않고 하나의 <code>errors</code> 객체로 통합(5절 참고)
댓글 입력창의 임시 텍스트 (<code>draft</code>)	<code>CommentForm</code>	등록/수정 모드 전환 시에도 입력창 자체는 이 컴포넌트가 소유. <code>editingComment</code> prop이 바뀌면 <code>useEffect</code> 로 프리필
이미지 파일 미리보기 URL	각 업로더 컴포넌트 또는 폼 <code>values.imageFile</code> 에서 파생 계산	<code>URL.createObjectURL</code> 결과를 별도 state로 저장하지 않고 <code>useMemo</code> 로 파생시키는 것을 권장(메모리 해제 누락 방지)

4. API 호출 위치와 데이터/이벤트 흐름

원칙

- API 호출은 그 데이터를 "소유"하는 가장 가까운 상위 컴포넌트(주로 Page 또는 그 Page 전용 커스텀 훅)에서만 수행합니다. 표시만 담당하는 하위 컴포넌트(`PostCard`, `CommentItem`, `ValidatedField` 등)는 API를 직접 호출하지 않습니다.
- 데이터는 항상 **props**로 아래로 흐르고, 사용자 상호작용(클릭/입력/제출)은 항상 **콜백 함수**로 위로 전달됩니다. React의 단방향 데이터 흐름 원칙 그대로입니다.

Board — `useInfinitePosts`

- `BoardPage` 가 `useInfinitePosts()` 훅을 호출 → 훅 내부에서 `IntersectionObserver` 가 `ScrollSentinel` 의 교차를 감지하면 `GET /posts?cursor=...` 호출, `cursorId / isLoading / posts` 갱신.
- 데이터 흐름: `useInfinitePosts` → `posts` 배열 → `BoardPage` → `PostList` → `PostCard` (개별 post 객체). `PostCard` 에서 발생하는 이벤트는 클릭 시 라우터 이동뿐이라 콜백이 필요 없고, `<Link>` 로 처리.

Post Detail — `usePostDetail(postId)`

- 이 훅 하나가 8개 API(`getDetailPost`, `deletePost`, `createComment`, `editComment`, `deleteComment`, `likePost`, `unlikePost`, `reportPost`)를 모두 소유합니다(기존 `post_detail.js` 가 파일 하나에 8개를 몰아 둔 구조를 훅 단위로 그대로 옮김).
- 데이터 흐름(아래로): `post`, `comments`, `isLiked`, `likeCount`, `commentCount`, `editingComment`, `deleteTargetCommentId` → `PostDetailPage` → 각 하위 컴포넌트(`PostHeader`, `LikeButton`, `CommentList`, `CommentForm` 등)로 필요한 부분만 분해해서 전달.
- 이벤트 흐름(위로):
 - `CommentItem` 의 수정 버튼 클릭 → `onEdit(comment)` → `PostDetailPage` 가 `editingComment` state를 설정 → `CommentForm` 이 그 값을 prop으로 받아 입력창을 프리필하고 버튼 라벨을 "댓글 수정"으로 변경.
 - `CommentForm` 제출 → `onSubmit(text)` → `PostDetailPage` 가 `editingComment` 유무로 `editComment` 또는 `createComment` 중 어떤 API를 부를지 분기 → 성공 시 `comments` 배열을 갱신(불변 업데이트: 수정은 `map`, 생성은 `[newComment, ...prev]`, 삭제는 `filter`) → React가 자동으로 리렌더링(기존 `diff / patch / rerenderComments` 수동 호출이 전부 사라짐).
 - `CommentItem` 의 삭제 버튼 클릭 → `onRequestDelete(commentId)` → `deleteTargetCommentId` state 설정 + 모달 `open=true` → `ConfirmModal` 의 확인 클릭 → `PostDetailPage` 가 `deleteComment(deleteTargetCommentId)` 호출 → 성공 시 `comments` 에서 filter.
 - `LikeButton` 클릭 → `onToggle()` → `PostDetailPage` 가 현재 `isLiked` 값을 보고 `likePost / unlikePost` 중 호출 → 응답의 `like_count` 로 상태 갱신.

Post Write / Post Edit

- `PostWritePage / PostEditPage` 가 각각 `writePost / editPost` (+ 수정은 진입 시 `defaultEditPage`) API를 소유.
- `PostForm` 은 `values` (제어 상태)만 들고 있다가 제출 버튼 클릭 시 `onSubmit(values)` 로 Page에 전달. Page가 `FormData` 구성 + API 호출 + 성공 후 라우팅까지 담당.
- Edit 모드는 Page가 마운트 시 조화한 값을 `initialValues` prop으로 `PostForm` 에 내려주고, `PostForm` 은 그 값으로 로컬 `values` 를 초기화.

Profile Edit / Password Edit

- `ProfileEditPage` 가 `getUser / updateUser / withdrawUser` 3개 API를 모두 소유(기존 `profile_edit.js` 가 `withdrawUser / getUser` 만 `request.js` 를 우회해 직접 `fetch` 를 쓰던 부분은 이 계층 정리 과정에서 반드시 `request` 공용 함수로 통일 — 5절 참고).
- `PasswordEditPage` 가 `passwordEdit` API 소유. 기존 코드의 미정의 `userId` 참조 버그(분석 자료 5절)는 애초에 요청 바디에 `userId` 를 넣지 않는 것이 맞으므로(서버가 토큰에서 식별), React 이전 시점에 제거.

Login / Signup

- 각 Page가 `login / signUp` API 호출 소유. 성공 시 `response.data.link` 로 이동, 로그인 성공 시 `AuthContext` 에 `accessToken` 저장(→ `localStorage` 에도 동기화해 새로고침 후에도 유지).
- 폼 자체(`LoginForm / SignupForm`)는 `onSubmit(values)` 콜백만 노출.

5. 기존 vanilla JS 상태 관리 대비 React에서의 개선점

기존 방식 (vanilla)	문제/한계	React 전환 후
모듈 스코프 <code>let</code> 전역 변수(<code>cursorId</code> , <code>isLoading</code> , <code>isEditing</code> , <code>currentEditCommentId</code> 등)	값이 바뀌어도 화면이 자동으로 갱신되지 않아, 변경할 때마다 관련 DOM 조작 코드를 수동으로 같이 호출해야 함(빠뜨리면 화면과 상태가 어긋남)	<code>useState</code> / 커스텀 훅으로 승격 → <code>set</code> 함수 호출만으로 리렌더링이 보장됨. 상태와 화면이 어긋날 수 없음
<code>data-comment-id</code> 속성 + 이벤트 위임으로 클릭된 댓글 식별	DOM이 사실상 "댓글 ID 저장소" 역할을 겸함 — 상태가 JS 변수가 아니라 마크업에 숨어 있어 추적이 어려움	<code>CommentItem</code> 이 <code>comment</code> 객체를 클로저로 이미 갖고 있으므로 <code>onEdit(comment)</code> 처럼 객체 자체를 콜백 인자로 전달. <code>data-*</code> 속성으로 우회할 필요가 없어짐
<code>classList.add / remove</code> 로 <code>active / liked / collapsed / show</code> 등의 상태 표현	"지금 좋아요 상태인가?"를 알려면 DOM class를 다시 읽어야 함 (<code>likeBtn.classList.contains('liked')</code>) , <code>post_detail.js:404</code>) — 상태의 원천이 DOM	<code>liked</code> , <code>collapsed</code> 등은 boolean state가 원천이 되고, <code>className</code> 은 <code>liked ? 'active' : ''</code> 처럼 그 상태의 파생값으로만 계산
댓글 목록: 최초엔 <code>innerHTML / appendChild</code> 1회성 렌더 → 이후 직접 만든 <code>mini-vdom(diff / patch)</code> 으로 재렌더	배열 상태 변경 → 재렌더링 트리거를 직접 구현/유지보수해야 함(이 저장소의 <code>vdom/</code> 4개 파일 전체가 이 문제 하나를 위한 코드)	<code>comments</code> state 배열만 불변 업데이트(<code>map/filter/spread</code>)하면 React가 재조정(<code>reconciliation</code>)을 알아서 수행 — <code>diff / patch / mountComments / rerenderComments</code> 같은 수동 파이프라인이 통째로 필요 없어짐
로그아웃 버튼: <code>Profile_edit.js</code> 에만 이벤트 리스너 등록, 나머지 5개 페이지는 HTML만 있고 리스너 없음	페이지마다 JS 파일이 독립이라 같은 마크업이라도 로직을 각자 다시 붙여야 하고, 빠뜨려도 아무 에러 없이 조용히 무반응	<code>Header</code> 공용 컴포넌트 하나에만 로그아웃 로직을 구현하고 모든 페이지가 <code>AppLayout</code> 을 통해 재사용 → 구조적으로 빠를 수 없음(버그가 아키텍처 차원에서 제거됨)
<code>Toast</code> /포맷 함수 (<code>formatCount</code> , <code>formatDateTime</code>)/검증 정규식이 파일마다 복붙	한 곳을 고치면 나머지 복사본은 그대로 남아 동작이 어긋날 위험(예: 정규식 4곳 중 하나만 수정)	공용 유틸(<code>validators.js</code> , <code>format.js</code>) + 공용 컴포넌트/훅(<code>Toast</code> , <code>useToast</code>)으로 단일화
필드별 <code>isValidEmail / isValidPassword / ...</code> 불리언을 따로 두고 검증할 때마다 수동으로 <code>activeXButton()</code> 호출	검증 로직과 "버튼 활성화 여부 재계산"을 매 핸들러마다 나란히 호출해야 함 — 하나라도 호출을 빠뜨리면 버튼 상태가 실제 값과 불일치	폼 전체를 하나의 <code>values + errors</code> 객체로 관리하고, 제출 버튼의 <code>disabled</code> 는 <code>Object.values(errors).some(Boolean)</code> 처럼 파생 계산값으로 처리 → 별도 동기화 호출이 필요 없음
<code>localStorage.getItem('accessToken')</code> 을 <code>request.js</code> 뿐 아니라 여러 페이지가 직접 읽음, <code>Profile_edit.js</code> 는 아예 <code>request.js</code> 를 우회해 <code>fetch</code> 를 직접 호출(공통 에러 처리 미적용)	인증 상태의 "원천"이 여러 곳에 흩어져 있어 정책(토큰 만료 처리 등) 변경 시 모든 참조 지점을 찾아 고쳐야 함	<code>AuthContext</code> 하나가 <code>accessToken</code> 을 소유하고 API 레이어가 이를 참조 → 모든 요청이 동일한 인증/에러 처리 경로를 강제로 통과
페이지 간 상태 전파가 전역 스토어 없이 URL 쿼리 (<code>?postId=</code>) + 페이지 재진입 시 서버 재조회로만 이루어짐	SPA 전환 시 "재조회 시점"을 명시적으로 관리해줘야 함(그냥 두면 안 됨)	<code>usePostDetail(postId)</code> 같은 훅이 <code>postId</code> (React Router 파라미터)가 바뀔 때마다 재조회하도록 <code>useEffect</code> 의존성 배열로 명시적으로 관리 — 기존과 같은 "URL 기반 재조회" 계약을 유지하면서도 그 시점을 코드로 드러냄

부록 — mini-vdom이 다루던 댓글 상태 → React 대응표

`Page/Post_detail/vdom/` (`createElement/diff/patch/mount/render`)이 실제로 관리하던 상태와, 그 상태가 바뀌는 시점을 그대로 표로 옮깁니다. React에서는 아래 상태들이 모두 `usePostDetail` 훅(또는 `PostDetailPage`) 안의 `useState` 로 대체되고, 재렌더링 트리거 역할을 하던 `mountComments()` / `rerenderComments()` 호출은 전부 삭제됩니다(상태 변경 자체가 곧 재렌더 트리거).

기존 vdom 코드의 상태/변수	기존 갱신 시점	React 대응 상태	갱신 방식
<code>let comments = []</code> (<code>post_detail.js:127</code>)	게시글 상세 조회 응답, 댓글 생성/수정/삭제 성공 시	<code>comments</code> (<code>usePostDetail</code> 내부 <code>useState</code>)	조회: <code>setComments(res.data.comments)</code> 생성: <code>setComments(prev => [...prev, newComment])</code> / 수정: <code>setComments(prev => prev.map((comment, index) => index === id ? newComment : comment))</code> 삭제: <code>setComments(prev => prev.filter(comment => comment.id !== id))</code>
<code>isEditing</code> (<code>post_detail.js:56</code>)	댓글 수정 버튼 클릭 시 <code>true</code> , 제출 완료 시 <code>false</code>	별도 boolean 대신 <code>editingComment !== null</code> 로 파생	<code>editingComment</code> state 하나로 통합(이 없으면 곧 "작성 모드")

기존 vdom 코드의 상태/변수	기존 갱신 시점	React 대응 상태	갱신 방식
currentEditCommentId / currentEditCommentBody (post_detail.js:195~196)	수정 버튼 클릭 시 설정, 제출 완료 시 null 로 초기화	editingComment (댓글 객 체 또는 null)	CommentItem 의 onEdit(comment) setEditingComment(comment) . 지 후 setEditingComment(null)
currentDeleteCommentId / currentDeleteItem (post_detail.js:198~199)	삭제 버튼 클릭 시 설정, 모달 취소/확인 시 초 기화	deleteTargetCommentId (숫자 또는 null)	CommentItem 의 onRequestDelete(commentId) → setDeleteTargetCommentId(comment currentDeleteItem (DOM 참조)은 불필요 — comments.filter 가 대상 체
commentList._prevVnode / _domNode (post_detail.js:256~ 257, 297, 300, 303)	mountComments / rerenderComments 호출마다	대응 상태 없음	React의 내부 fiber 트리가 이 역할을 완전 애플리케이션 코드에서 "이전 트리"를 직접 을 필요가 없음
commentCountHeading.textContent (댓글 생성/삭제 응답의 commentCount)	댓글 생성/삭제 성공 시	commentCount (usePostDetail 내부 state)	API 응답의 result.data.commentC setCommentCount(...) , CommentCountHeading 은 이 값을 2
comment.commentId 를 vnode의 key 로 사용(post_detail.js:219 , "key 필 수!" 주석)	vdom diff가 리스트 아이템을 식별하기 위해	<CommentItem key= {comment.commentId} .../>	동일한 이유(리스트 재조정 시 아이템 식별 에서도 그대로 유지해야 하는 값 — vdom React로 넘어가도 "댓글은 commentId를 별한다"는 설계는 변하지 않음